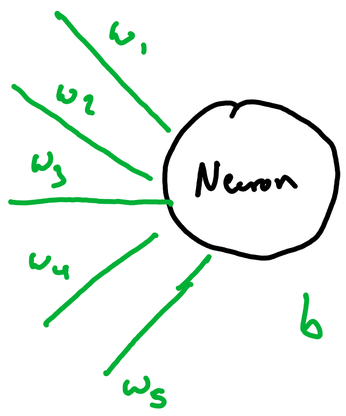
The Mathematics For Neural Networks

A neural network consists of layers of neurons, inspired by the human brain. We make use of these, with a lot of calculus to create learning algorithms.

Neurons:

A neuron takes in an input vector, calculates a weighted sum of the inputs, then adds a scalar bias. We tune these weights and biases until the final output does what we want it to.



We call the weights and biases the parameters.

We collect the weights into a vector.

$$\underline{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$$

So mathematically, for an input vector $\underline{x}$, the output of the neuron is defined as

$$n = \underline{w}^T \underline{x} + b$$

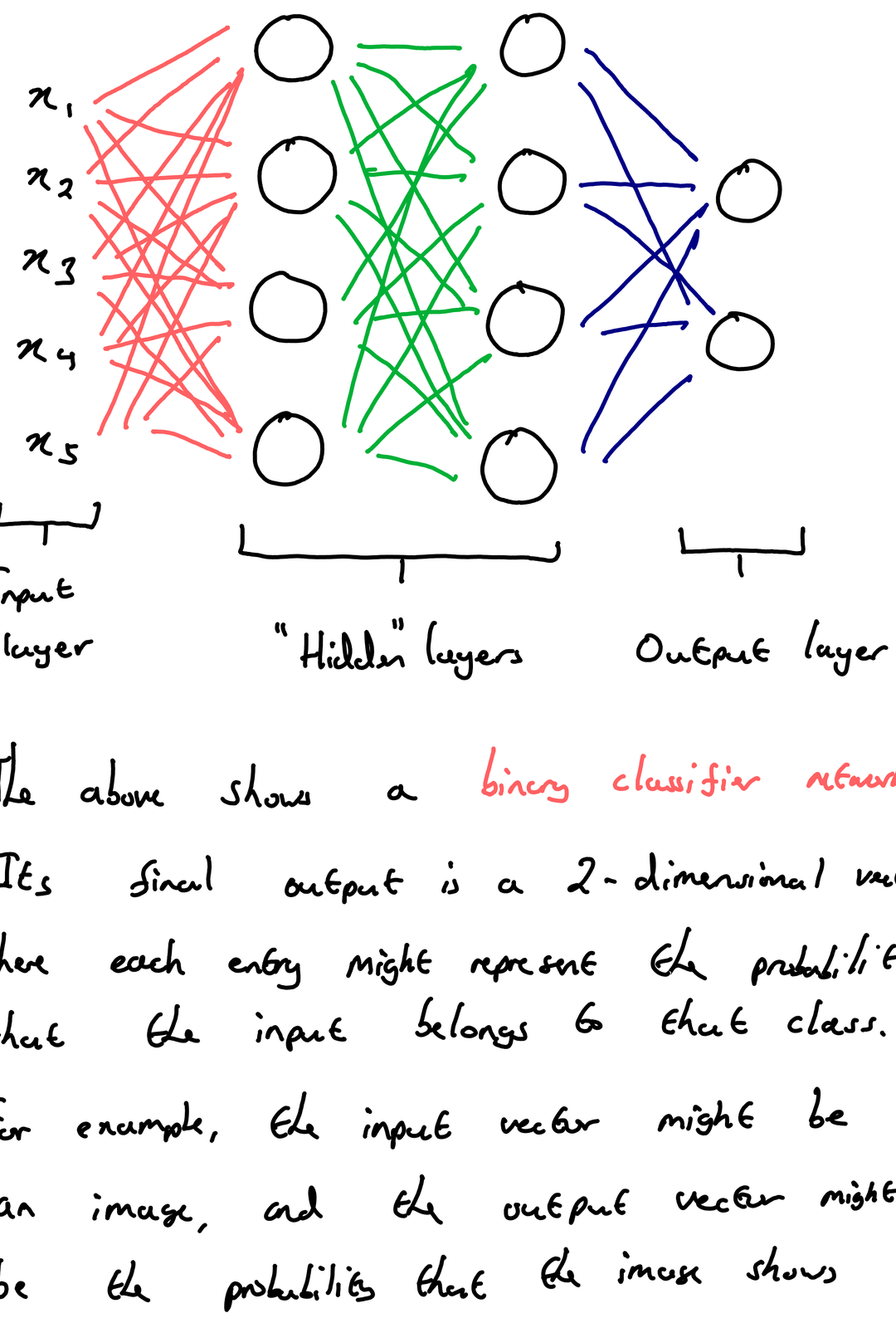
where $a^T b$ gives the dot product of two vectors. We also require an activation function that is non linear. If we did not have this, then composing linear neurons would be the same operation as using one big neuron, and we don't get any interesting (or useful) networks. For now we refer generally to our activation function as $\sigma(n)$, and we can choose it later. A common one is $\sigma(n) = \tanh(n)$

So our activated neuron is given by

$$N = \sigma(\underline{w}^T \underline{x} + b)$$

Layers of Neurons:

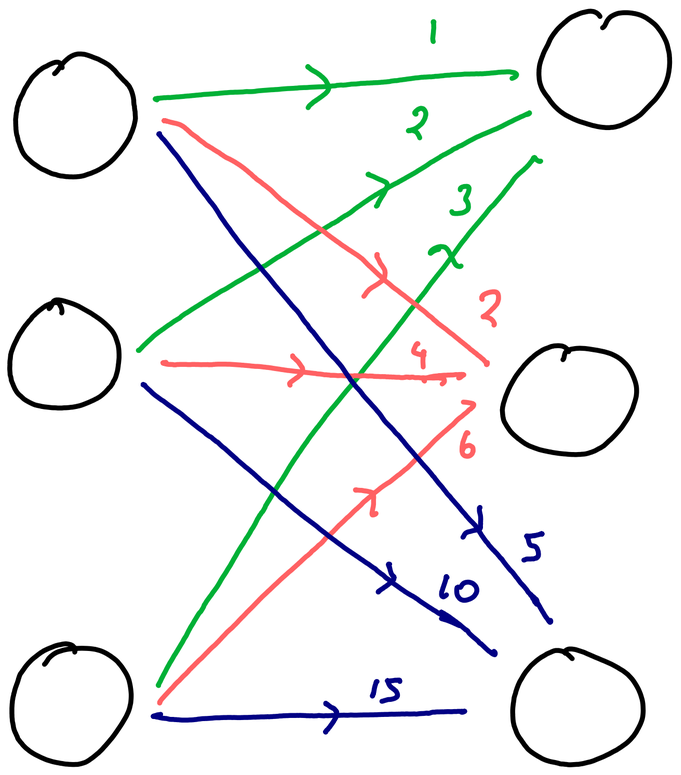
Now that we understand the structure of a single neuron, we want to layer them. Let's look at an example:



The above shows a binary classifier network. Its final output is a 2-dimensional vector, where each entry might represent the probability that the input belongs to that class. For example, the input vector might be an image, and the output vector might be the probabilities that the image shows a cat, and the probabilities it shows a dog. (you would of course choose the higher probabilities to make your classification)

Now, how do we extend our mathematical framework of the neuron to handle these layers? Well, the weights vector now becomes a weights matrix and the bias scalar now becomes a bias vector

Consider this very simple case:



This shows a simple network with the connections labelled and colour coded. Note that each neuron in layer i has a connection to every neuron in layer $i+1$. We construct the weights matrix W such that $w_{i,j}$ holds the weight from the ith neuron in the current layer to the jth neuron in the next.

$$W = \begin{bmatrix} 1 & 2 & 5 \\ 2 & 4 & 10 \\ 3 & 6 & 15 \end{bmatrix}$$

$w_{3,1}$ - The connection from the 3rd neuron in the current layer to the 1st neuron in the next layer.

So we see that each column represents the input to the corresponding neuron in the next layer. So logically extending from the single neuron, the output of a layer is given by

$$z = W^T x + b$$

Then activated:

$$z = \sigma(W^T x + b)$$

where σ is performed elementwise.

So z is a vector where the kth entry contains the output of the kth neuron.

To demonstrate:

$$W = \begin{bmatrix} | & | & | \\ w_1 & w_2 & w_3 \\ | & | & | \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

$$W^T x + b$$

$$\begin{bmatrix} - & w_1^T & - \\ - & w_2^T & - \\ - & w_3^T & - \end{bmatrix} x + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

$$\begin{bmatrix} w_1^T x + b_1 \\ w_2^T x + b_2 \\ w_3^T x + b_3 \end{bmatrix}$$

So our activated layer is:

$$z = \sigma(W^T x + b)$$

$$= \begin{bmatrix} \sigma(w_1^T x + b_1) \\ \sigma(w_2^T x + b_2) \\ \sigma(w_3^T x + b_3) \end{bmatrix}$$

So we clearly see that each entry is the output of the ith neuron.

Gradients:

Now, our goal is to tune all the weights and biases so that the output of the model is useful and not just a collection of random numbers. To do this, we must first define a Loss function.

The loss function is a single number that encapsulates how wrong the models prediction /output is. It is some measure of the difference between the models output and the correct answer. Our goal is to tweak the weights and biases until our loss function is minimised, through the use of calculus and an algorithm called Gradient Descent, which we will cover later.

We want to calculate two quantities:

$$\frac{\partial \ell}{\partial W} \quad \text{and} \quad \frac{\partial \ell}{\partial b}$$

The derivative of the loss w.r.t the weights and biases. We can then use this to follow the gradient until the loss ℓ is minimised. This will involve heavy use of the chain rule so let's build up intuition from the neuron upwards.

Derivatives of Neurons:

If you recall, our activated neuron is given by

$$N = \sigma(\underline{w}^T \underline{x} + b)$$

How does the output of this neuron change if we perturb $\underline{w}$ or b slightly? We are asking, what is $\partial N/\partial \underline{w}$ and $\partial N/\partial b$?

Derivative w.r.t weights

Let's make the substitution $z = \underline{w}^T \underline{x} + b$

So:

$$N = \sigma(z)$$

Thus:

$$\frac{\partial N}{\partial \underline{w}} = \frac{\partial N}{\partial z} * \frac{\partial z}{\partial \underline{w}}$$

$$\frac{\partial N}{\partial z} = \sigma'(z)$$

(Closed form depends on what activation function was used)

$$z = \underline{w}^T \underline{x} + b$$

$$= w_1 x_1 + w_2 x_2 + \dots w_n x_n + b$$

Now, $\underline{w}$ is a vector, so when we differentiate a scalar w.r.t a vector, we get a vector out, where the kth entry is the expression differentiated w.r.t the kth element of the vector.

$$\frac{\partial z}{\partial \underline{w}} = \begin{bmatrix} \partial z/\partial w_1 \\ \partial z/\partial w_2 \\ \partial z/\partial w_3 \\ \vdots \\ \partial z/\partial w_n \end{bmatrix}$$

$$= \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \underline{x}$$

So plugging back in:

$$\frac{\partial N}{\partial \underline{w}} = \frac{\partial N}{\partial z} * \frac{\partial z}{\partial \underline{w}}$$

$$= \sigma'(z)\, \underline{x}$$

$$= \sigma'(\underline{w}^T \underline{x} + b) \odot \underline{x}$$

$\odot$ ← elementwise multiplication

Derivative w.r.t Bias:

By a very similar process:

$$\frac{\partial N}{\partial b} = \frac{\partial N}{\partial z} * \frac{\partial z}{\partial b}$$

$$= \sigma'(z)\frac{\partial z}{\partial b}$$

$$= \sigma'(z) \times 1$$

$$= \sigma'(z)$$

$$= \sigma'(\underline{w}^T \underline{x} + b)$$

Gradients of Layers:

We want to find $\partial L/\partial W$ and $\partial L/\partial \underline{b}$.

Let's first think about the weights.

We immediately run into an issue, L is a vector and W is a matrix. Thus $\partial L/\partial W$ is a third order tensor.

This isn't a problem when it comes to computation, but it is very hard for humans to work with such structures intuitively. Thus, we go up a level and think about $\partial \ell/\partial W$, the derivative of the loss w.r.t the weights. The loss is a scalar, so $\partial \ell/\partial W$ is a matrix the same size as W. We will use the Mean Squared Error (MSE) as our loss function.

$$\ell = \frac{1}{N}\sum_{i=1}^{N} \|y_i - \hat{y}_i\|^2$$

where

y_i is the correct answer for the ith input

$\hat{y}_i$ is the model output for the ith input

$\|y_i - \hat{y}_i\|^2$ is the squared norm of the difference between the two.

So the loss is the squared average of the difference between model output and correct answer for every input.

Starting with derivative w.r.t weights:

$$\frac{\partial \ell}{\partial W} = \frac{1}{N}\sum_{i=1}^{N} \frac{\partial}{\partial W}\left(\|y_i - \hat{y}_i\|^2\right)$$

So we only really need to think about

$$\frac{\partial}{\partial W}\left(\|y - \hat{y}_i\|^2\right)$$

Using $\underline{u} = y - \hat{y}_i$

$$\frac{\partial}{\partial W}\|\underline{u}\|^2 = \frac{\partial}{\partial \underline{u}}\|\underline{u}\|^2 \times \frac{\partial \underline{u}}{\partial W}$$

$$= 2\underline{u} \times \frac{\partial \underline{u}}{\partial W}$$

Now $\hat{y}_i$ is defined recursively:

$$\hat{y}_i = \sigma\left(W_i^T L_{i-1} + \underline{b}_i\right)$$

(The output of the last layer is the input of the current layer)

So

$$\underline{u} = y - \sigma\left(W_i^T L_{i-1} + \underline{b}_i\right)$$

Let $\underline{z} = W_i^T L_{i-1} + \underline{b}_i$

$$\underline{u} = y - \sigma(\underline{z})$$

$$\frac{\partial \underline{u}}{\partial \underline{z}} = -\sigma'(\underline{z})$$

So now the only missing piece is $\partial \underline{z}/\partial W$

Since:

$$\frac{\partial \ell}{\partial W} = \frac{\partial \ell}{\partial \underline{u}} \cdot \frac{\partial \underline{u}}{\partial \underline{z}} \times \frac{\partial \underline{z}}{\partial W}$$

$$= 2\underline{u} \times -\sigma'(\underline{z}) \cdot \frac{\partial \underline{z}}{\partial W}$$

$$= -2\underline{u} \odot \sigma'(\underline{z}) \frac{\partial \underline{z}}{\partial W}$$

For simplicity, we define δ to be

$$\delta = -2u \odot \sigma'(z)$$

Such that:

$$\frac{\partial \ell}{\partial W} = \delta \frac{\partial z}{\partial W}$$

We call δ the error vector. Now, we're run into the same problem as before: z is a vector and W a matrix, meaning $\frac{\partial z}{\partial W}$ is a tensor still. However, since we know that $\frac{\partial \ell}{\partial W}$ is a matrix, perhaps if we analyze it elementwise we can discover a pattern that will help us find a closed form for $\frac{\partial \ell}{\partial W}$. In other words, can we find an expression for the general entry

$$\frac{\partial \ell}{\partial W_{a,b}}$$

that makes the matrix form of $\frac{\partial \ell}{\partial W}$ clear? To do this, we must first understand the multivariable chain rule

Multivariable Chain Rule

Consider the single variable case:

$$\ell = f(z) \qquad z = g(\epsilon)$$

$$\Rightarrow \quad \ell = f(g(\epsilon))$$

Then

$$\frac{d\ell}{d\epsilon} = \frac{d\ell}{dz} \times \frac{dz}{d\epsilon}$$

Notably, there is only one pathway through ϵ that can affect ℓ. Now, what happens with vectors?

$$z = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_n \end{bmatrix}$$

Now each z_i depends on ϵ. So perturbing ϵ causes all z_i to change simultaneously. So the total effect on ℓ is the sum

$$\frac{\partial \ell}{\partial \epsilon} = \sum_k \frac{\partial \ell}{\partial z_k} \cdot \frac{\partial z_k}{\partial \epsilon}$$

So return to $\partial \ell / \partial W_{a,b}$. The only pathway to W is through z, so the multivariable chain rule applies:

$$\frac{\partial \ell}{\partial W_{a,b}} = \sum_k \frac{\partial \ell}{\partial z_k} \cdot \frac{\partial z_k}{\partial W_{a,b}}$$

But note that we defined the error vector as:

$$\delta = \frac{\partial \ell}{\partial u} \times \frac{\partial u}{\partial z}$$

$$= \frac{\partial \ell}{\partial z}$$

So $\frac{\partial \ell}{\partial z_k}$ is the kth component of the error vector!

$$\frac{\partial \ell}{\partial W_{a,b}} = \sum_k \delta_k \frac{\partial z_k}{\partial W_{a,b}}$$

(δ_k: a scalar; kth element of δ)

Now let's evaluate $\frac{\partial z_k}{\partial W_{a,b}}$.

Since

$$z = W_i^T \mathcal{L}_{i-1} + b_i$$

$$z_k = \sum_m W_{mk} \cdot (\mathcal{L}_{i-1})_m + b_k$$

$$z_k = \sum_m W_{mk} \cdot (L_{i-1})_m + b_k$$

(repeated)

$W_{a,b}$ is a single, fixed entry. So when we differentiate z_k w.r.t $W_{a,b}$, the only terms that survive are the ones where $m=a$ AND $k=b$.

So:

$$\frac{\partial z_k}{\partial W_{a,b}} = \begin{cases} (L_{i-1})_a & \text{if } k=b \text{ and } m=a \\ 0 & \text{otherwise} \end{cases}$$

If we plug this into the chain rule expression, only the $k=b$ term survives

$$\frac{\partial \ell}{\partial W_{a,b}} = \delta_b (L_{i-1})_a$$

So the term at row a, column b is given by this expression. But notice that this is the same as the matrix multiplication

$$\frac{\partial \ell}{\partial W} = L_{i-1}\, \delta^T$$

Derivative w.r.t Bias

We perform a similar process for the derivative w.r.t the Bias:

$$\frac{\partial \ell}{\partial b} = \delta \frac{\partial z}{\partial b}$$

This time, $\partial z / \partial b$ is a matrix. Thinking about each entry $\frac{\partial z_k}{\partial b_j}$

$$z_k = \sum_m W_{mk} (L_{i-1})_m + b_k$$

We see:

$$\frac{\partial z_k}{\partial b_j} = \begin{cases} 1 & \text{if } k=j \\ 0 & k \neq j \end{cases}$$

So $\frac{\partial z}{\partial b} = I$.

So $\frac{\partial \ell}{\partial b} = \delta I = \delta$

Propagating

So what have we accomplished? We have found the derivatives for the final layer. (This is the output $\hat{y}_i$).

Can we use this to find the derivatives for all previous layers? What we are really asking is, how can we relate the loss of layer i to layer $i-1$? We have two pieces of information:

$$\frac{\partial \ell}{\partial W_i} = L_{i-1}\, \delta_i^T \quad \text{and} \quad \frac{\partial \ell}{\partial b_i} = \delta_i$$

Since both expressions involve δ, it is wise to target an expression that makes use of it, since it would save computation. So we want to somehow relate δ_i to $i-1$. If we know all δ, all $\frac{\partial \ell}{\partial W}$ and $\frac{\partial \ell}{\partial b}$ can be easily calculated. To find δ_{i-1} we can use the chain rule, since:

$$\delta_i = \frac{\partial \ell}{\partial z_i}$$

So:

$$\frac{\partial \ell}{\partial z_{i-1}} = \sum_k \frac{\partial \ell}{\partial z_k^{(i)}} \cdot \frac{\partial z_k^{(i)}}{\partial z^{(i-1)}}$$

(We must use the multivariable chain rule since $z^{(i)}$ and $z^{(i-1)}$ are vectors)

Now, $\frac{\partial \ell}{\partial z^{(i-1)}}$ is a vector, so it's wise to try to find a pattern elementwise. Consider

$$\frac{\partial \ell}{\partial z^{(i-1)}_j} = \sum_k \frac{\partial \ell}{\partial z^{(i)}_k} \cdot \frac{\partial z^{(i)}_k}{\partial z^{(i-1)}_j}$$

jth component of $\partial \ell / \partial z^{(i-1)}$

$$= \sum_k \delta^{(i)}_k \frac{\partial z^{(i)}_k}{\partial z^{(i-1)}_j}$$

We now need to compute the closed form of $\partial z^{(i)}_k / \partial z^{(i-1)}_j$. How does $z^{(i)}$ to $z^{(i-1)}$? If you recall:

$$z^{(i)} = W_i^T a_{i-1} + b_i$$

$$\Rightarrow z^{(i)} = W_i^T \sigma(z_{i-1}) + b_i$$

So

$$z^{(i)}_k = \sum_m (W_i)_{m,k} \sigma(z^{(i-1)}_m) + b_m$$

If we differentiate w.r.t $z^{(i-1)}_j$, only the term where $m = j$ survives.

$$z^{(i)}_k = (W_i)_{j,k} \sigma'(z^{(i-1)}_m)$$

Substituting back in:

$$\frac{\partial \ell}{\partial z^{(i-1)}_j} = \sum_k \delta^{(i)}_k \cdot (W_i)_{j,k} \sigma'(z^{(i-1)}_j)$$

Factoring out:

$$= \sigma'(z^{(i-1)}_j) \sum_k \delta^{(i)}_k (W_i)_{j,k}$$

Therefore:

$$\delta^{(i-1)} = \sigma'(z^{(i-1)}) \odot W_i \delta^{(i)}$$

So now we've constructed a recursive relationship that allows $\delta^{(i-1)}$ to be found if we know $\delta^{(i)}$. This is back propagation.

The Backpropagation Algorithm:

Now we put all this together into a full algorithm.

1. Forward Pass - Compute and store all $z^{(i)}$ and $a^{(i)}$
2. Compute δ at the output layer
3. Use the recurrence relation to compute all previous δ
4. At each layer, use δ to compute the weights and bias derivatives

Gradient Descent

Now that we have all $\frac{\partial \ell}{\partial W}$ and $\frac{\partial \ell}{\partial b}$ for every layer, unfurl and concatenate them into a large gradient vector ∇. Similarly turn all parameters into a vector θ. We now update all the parameters:

$$\theta_{n+1} = \theta_n - \alpha \nabla$$

We step in the direction of steepest decrease of the loss function, working to minimize it.

END